Parte 1 – Questões Teóricas

1. **ReactJS e NextJS: Renderização e Estratégias**
   a) Explique, com suas próprias palavras, as diferenças entre:
   - **Client-Side Rendering (CSR)**
   - **Server-Side Rendering (SSR)**
   - **Static Site Generation (SSG)**

   b) Em que cenários cada uma dessas abordagens pode ser vantajosa ou problemática? Dê exemplos práticos.

2. **Backend: Conceitos e Approaches**
   a) Quais são as principais maneiras de implementar um backend para uma aplicação web?
   b) Conhece a lib tRPC? Se sim, quais são as principais vantagens e desvantagens de utilizar tRPC, especialmente em termos de segurança, tipagem e escalabilidade?

Parte 2 – Desafio Prático

Dica: Menos é mais, faça algo simples porém funcional!

Contexto:

Você deverá criar uma aplicação NextJS (v15) que simula um sistema simples de gerenciamento de tarefas. O backend será responsável por manter a lista de tarefas em memória (ou seja, não há necessidade de persistência em banco de dados) e será exposto via tRPC.

Requisitos do Desafio:

1. **Configuração do Projeto:**
   - Crie um projeto NextJS utilizando as versões mais recentes.
   - Configure o tRPC de forma que o frontend consiga consumir os endpoints definidos no backend.
2. **Backend (tRPC):**
   - **Modelagem:** Cada tarefa deve ter pelo menos os seguintes campos:
     - `id` (único, pode ser gerado automaticamente)
     - `titulo` (string, obrigatório)
     - `descricao` (string, opcional)
     - `dataCriacao` (timestamp)
   - **Operações CRUD:** Implemente via tRPC as seguintes operações:
     - **Criar Tarefa:** Endpoint para adicionar uma nova tarefa.
     - **Listar Tarefas:** Endpoint para retornar a lista de tarefas.

        - **Atualizar Tarefa:** Endpoint para editar os dados de uma tarefa existente.
        - **Deletar Tarefa:** Endpoint para remover uma tarefa.
    - **Validação e Tratamento de Erros:** Garanta que:
        - Tarefas não sejam criadas sem título.
        - As operações retornem erros significativos em caso de falha (por exemplo, tentativa de atualizar uma tarefa inexistente).
3. **Frontend (ReactJS/NextJS):**
    - **Página de Listagem de Tarefas:**
        - Utilize Server-Side Rendering (SSR) para pré-carregar a lista de tarefas.
        - Exiba as tarefas em um layout simples (não é necessário um design elaborado).
        - Permita a exclusão de tarefas diretamente na listagem, com feedback visual para o usuário (ex: mensagem de sucesso/erro).
    - **Página de Criação/Atualização de Tarefas:**
        - Crie um formulário para inserir os dados de uma nova tarefa ou editar uma existente.
        - Utilize componentes funcionais e hooks para gerenciar o estado do formulário.
        - Implemente validação no formulário (por exemplo, impedir o envio se o campo de título estiver vazio).
    - **Feedback e Tratamento de Erros:**
        - Exiba mensagens apropriadas em caso de erro durante as operações (tanto vindos do backend quanto da lógica de frontend).
        - Considere a experiência do usuário ao lidar com estados de carregamento, sucesso e falha.
4. **Extras (Bônus):**
    - **Infinite Scroll:** Na página de listagem, implemente uma funcionalidade de carregamento incremental das tarefas conforme o usuário rola a página.
    - **Comentários/Documentação:** Adicione comentários explicando suas principais decisões e, se possível, inclua um README descrevendo brevemente como o sistema funciona e como executá-lo.

Instruções de Entrega

- Mande um link de um repositório público em github.
- **IMPORTANTE: Não esqueça que o repositório será público, por isso certifique que não está colocando NENHUMA informação sensível, como informações pessoais, senhas, e tokens de APIs, essa parte é por sua conta e risco. TODAS AS INFORMAÇÕES PRECISAM SER PURAMENTE FICTÍCIAS.**